# Introduction on UML for Industrial Systems

Ansgar Radermacher / Asma Smaoui
ansgar.radermacher@cea.fr

**Acknowledgments** – contains material from my CEA colleagues Shuai Li, Jérémie Tatibouët, François Terrier, Sébastien Gérard

# Code generation from models (Model to Text)

1. **Code generation from models (Model to Text)**
2. **Role of static profiles**
3. **Language xtend**
4. **Apply to simplified Python/Keras generator**
5. **Test/debug an Eclipse plugin**
6. **Revision / questions**
7. **Homework**

# Code generation from Models

- Motivation
  - Model = *executable* specification (not only documentation)
  - Provide (limited) early feedback
  - Map model elements to code

- Variants
  - Generate skeleton vs. generate full code
  - Synchronize model from code (roundtrip) vs. Code generation only

# Recap: M2M and M2T transformations

- M2M = model to model transformation
  - Transform a source model into a target model

- Examples

  UML into another UML model (refinement)

  UML to an Ecore model

  …

# Model to text (M2T) transformation

- Produce textual artefacts from a model, typically
  - Code
  - Formal verification languages
  - Documentation
  - Textual description languages (such as IDL)

- Here: only focus on code generation

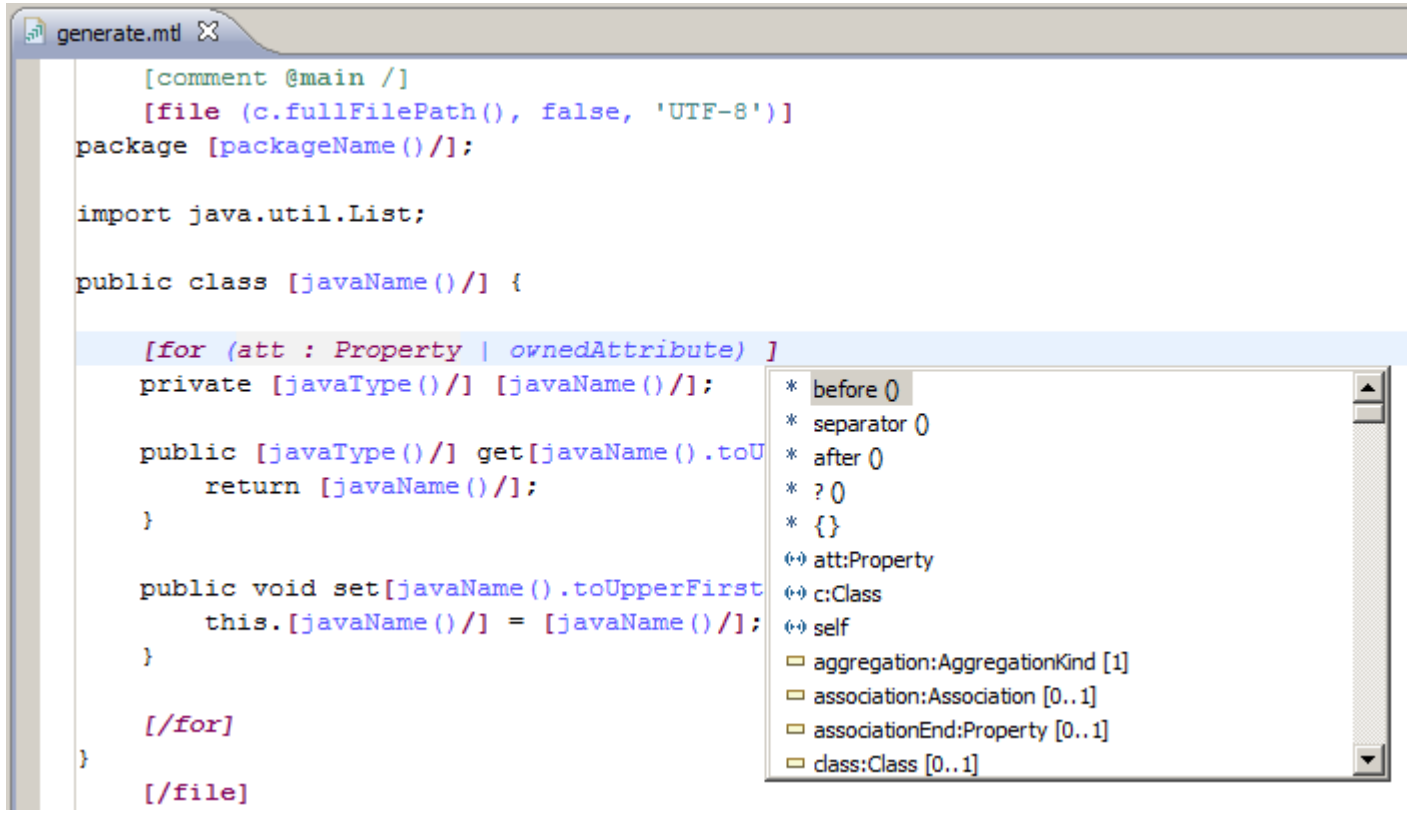
# Implementation approaches

- Use QVT or ATL
- Papyrus SW designer
  - Combination of M2M and M2T transformations
    (quite simple code generators, advanced features via M2M)
- OMG M2T (https://www.omg.org/spec/MOFM2T)
- Specific languages – Acceleo and Xtend (more on Xtend later)

# Acceleo

- “Generate anything from any EMF model”
- R&D result / support from OBEO

- Pragmatic
- Support for text templates
- Editor with assistance
- Interpreted language



# Code generation from models (Model to Text)

1. Code generation from models (Model to Text)
2. **Role of static profiles**
3. Language xtend
4. Apply to simplified Python/Keras generator
5. Test/debug an Eclipse plugin
6. Revision / questions

# Static profiles - Introduction

- We defined a profile, now need programmatic access to its **attributes**

- “Normal” profile – access attributes via generic methods; provided by common superclass `Element`

  - `getAppliedStereotype(“stereotype-name”) : Stereotype`
  - `getValue(stereotype, “attribute-name”) : Object`

# Standard profiles – Attribute access

```
Stereotype conv1D =
     l1.getAppliedStereotype(”SimpleNN::Conv1D”);
int filters = (int) l1.getValue(conv1D, ”filters”);
```

# Quiz

Disadvantage of generic stereotype access methods?

Stereotype or attribute name misspelled/renamed
⇨ code still compiles, runtime error

Generic method returns `Object`
⇨ need cast, no type safety!

# Static profiles - Introduction

- We defined a profile, now need programmatic access to its **attributes**
- “Normal” profile – access attributes via generic methods
  - umlElement.getStereotype(“stereotype-name”)
  - umlElement.getValue(stereotype, “attribute-name”)
- Stereotype instance = Instance of a Java class?
- Ecore provides possibility to generate Java code from a (meta-) model

- Steps : UML => ecore model => code

# Generated code …

Generated interface reflects inheritance hierarchy

```
public interface Conv1D extends Layer {
   /**
    * Returns the value of the '<em><b>Filters</b></em>' attribute.
    * …
    * @generated
    */
   int getFilters();

   /**
    * Sets value of '{@link …SimpleNN.Conv1D#getFilters <em>Filters</em>}' attribute.
    * …
    * @generated
    */
   void setFilters(int value);
}
```

Getter/setter for attributes

# Static profiles – Attribute access

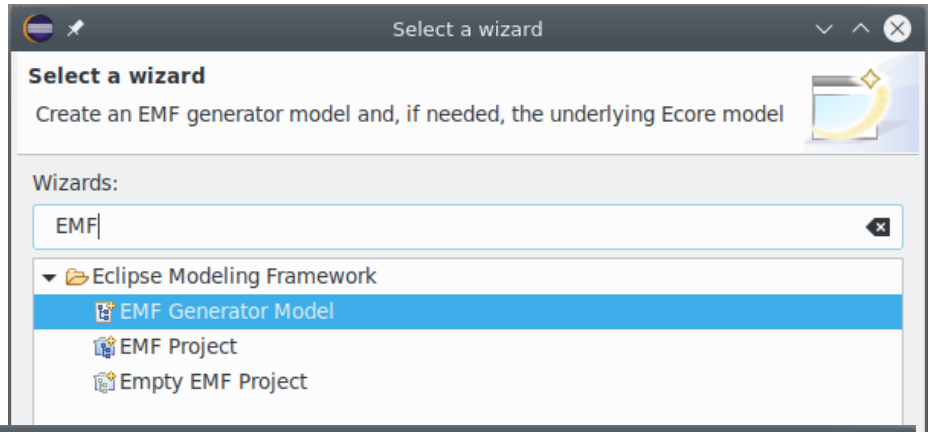
```
Stereotype conv1D =
      l1.getAppliedStereotype("SimpleNN::Conv1D");
int filters = (int) l1.getValue(conv1D, "filters");
```

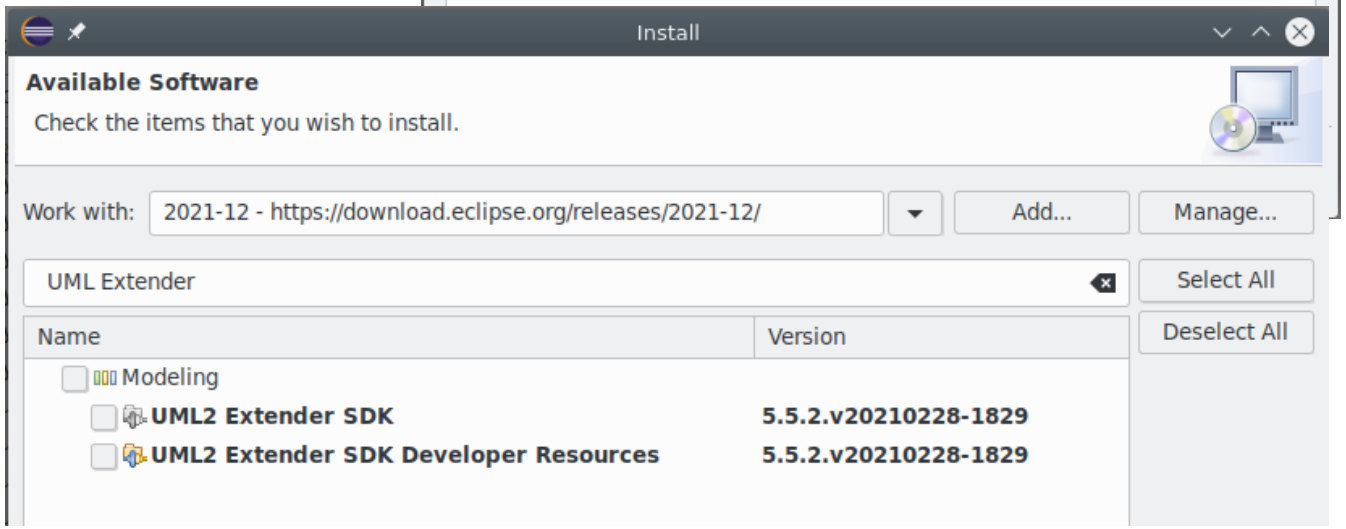
```
Conv1D conv1D =
      UMLUtil.getStereotypeApplication(l1, Conv1D.class);
int filters = conv1D.getFilters();
```

Type safe – return specific Conv1D interface

# Exercise – generate a static profile

- Download plugin `org.eclipse.papyrus.simplenn.profile` from git
- Import the plugin into Eclipse workspace
- In sub-folder `profile`
  - Create new EMF Generator model
  - Use name SimpleNN.profile.genmodel
  - Import option: UML model
  - Load model SimpleNN.profile.uml
  - Precondition install UML2 extender SDK



# Code generation from models (Model to Text)

1. Code generation from models (Model to Text)
2. Role of static profiles
3. **Language xtend**
4. Apply to simplified Python/Keras generator
5. Test/debug an Eclipse plugin
6. Revision / questions
7. Homework

# Xtend language

- Java dialect, compiles into (readable) Java 8 source code.
- Use existing Java libraries seamlessly.
- Macros, lambdas, operator overloading and many more modern language features.
- Type inference – no need to type in most case
- Text Templates (that´s why we´re looking at it)

# Type inference & syntactic sugar

- Replace
  `Type a = ...` with
  `var a = ...` or `val a =`

- The type with we infered from the expression on the right.
- Access “get” method as if an attribute – useful for generated MM methods
- Static extension, replace
  `UtilClass.fctA(a)` with
  `a.fctA`
- Clever indentation

# Xtend – text templates

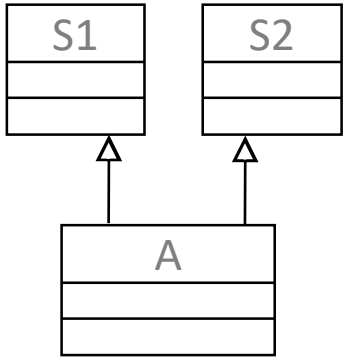
Shorthand for method getName()

Shorthand for method getSuperClasses()

- Example
- '''class «c.name» extends
  «FOR superclass : c. superclasses SEPARATOR “, “»
  «superclass.name»
  «ENDFOR»'''‘

FOR loop

Separator character

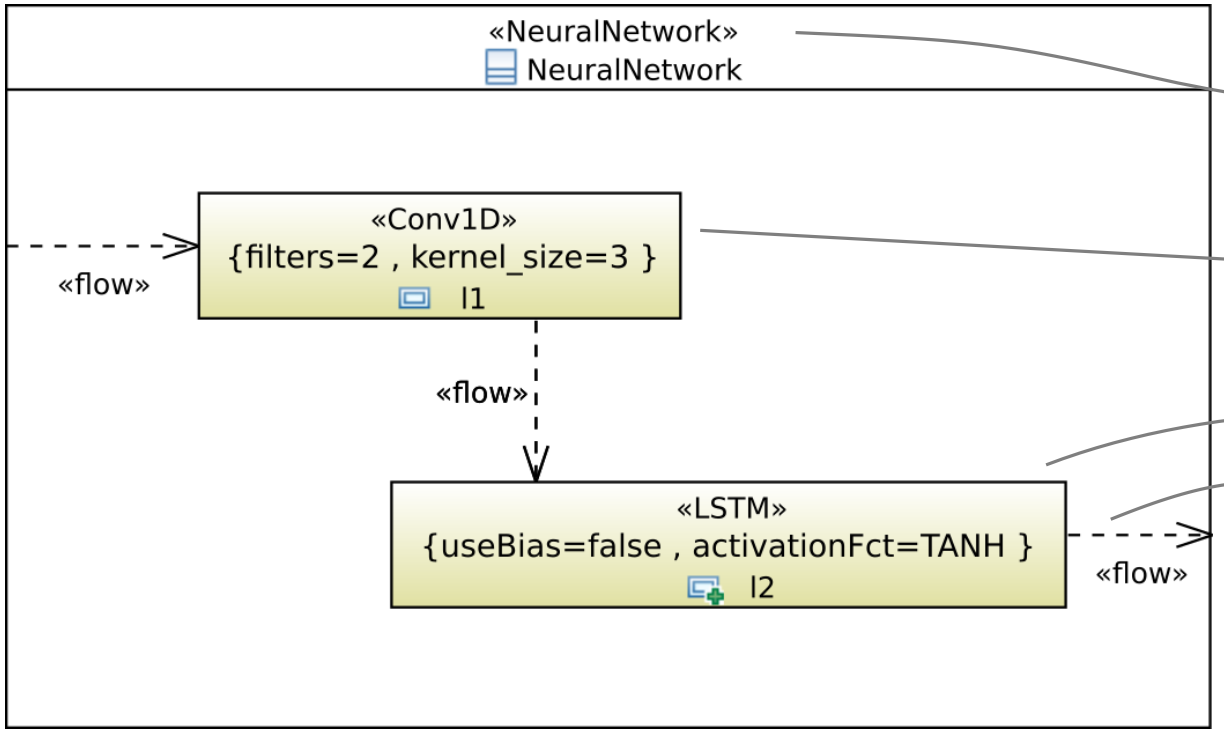



class A extends
S1, S2

# Code generation from models (Model to Text)

1. Code generation from models (Model to Text)
2. Role of static profiles
3. Language xtend
4. **Apply to simplified Python/Keras generator**
5. Test/debug an Eclipse plugin
6. Revision / questions
7. Homework

# Application for Neural Network




```python
import tensorflow as tf
from tensorflow import keras

class NeuralNetwork(tf.keras.Model):
    def __init_(self):
        super(module.base_Class.name», self).__init__();

        self.l1 = tf.keras.layers.Conv1D(2, 3);
        self.l2 = tf.keras.layers.LSTM(false, TANH);

    def call(self, inputs):
        x0 = self.l1(input)
        output = self.l2(x0)
        return output;

testModule = NeuralNetwork()
testModule.compile(name="NeuralNetwork")
```

**input model** (from cours2)
Stereotyped class with parts
composite structure diagram

**wanted output**
Python/Keras code

# Code generator, part 1 (xtend)

```
/**
 * Generate a model with different elements necessary for the compilation
 */
static def genModule(NeuralNetwork module) '''
  import tensorflow as tf
  from tensorflow import keras

  class «module.base_Class.name»(tf.keras.Model):
     def __init_(self):
        super(module.base_Class.name», self).__init__();

        «FOR attribute : module.base_Class.attributes»
              «val layer = UMLUtil.getStereotypeApplication(attribute, Layer)»
              self.«attribute.name» = «layer.genLayer»
        «ENDFOR»
```

Text template

Access to (meta-) model

Control loop (over attributes)

# Code generator, part 2 (xtend)

```
…
   def call(self, inputs):
      «var i = 0»
      «val flowList = module.base_Class.getFlows()»
      «FOR layer : flowList»
            «val previous = (i == 0) ? "input" : '''x«i-1»'''»
            «val current = (i == flowList.size-1) ? "output" : '''x«i++»'''»
            «current» = self.«layer.name»(«previous»)
      «ENDFOR»
      return output;

testModule = NeuralNetwork()
testModule.compile(name="NeuralNetwork")
```

Auxiliary function

input, intermediate, output

# Code generator, part 3 (xtend)

```
…
// Generate each layer according to the model
static def genLayer(Layer layer) {
   if (layer instanceof LSTM) {
      genLstm(layer as LSTM)
   } else if (layer instanceof Conv1D) {
      genConvolutionLayer(layer as Conv1D)
   }
}

// Generate convolution layer according to the model
static def genConvolutionLayer(Conv1D conv1d) '''
   tf.keras.layers.Conv1D(«conv1d.filters», «conv1d.kernel_size»);
'''
```

Use static profile

Access attributes of static profile

# Exercise

- Copy plugin `org.eclipse.papyrus.simplenn.gen.keras` into your workspace (and import it) – it´s on the git
- Some information about Eclipse plugins (a bit off-topic)
  - OSGI MANIFEST.MF – plugin properties, dependencies
  - plugin.xml - Extension points & properties (for menus, etc.)

# Acceleo vs. Xtend

- Text templates are relatively similar in both languages

- Advantages Acceleo
  - Dedicated to M2T, output file creation support
- Advantages Xtend
  - Compiled into Java => 1. Fast
    2. Use any Java function w/o specific declarations,
    3. Easy to debug with standard tooling
  - Standard language for code generators for Papyrus

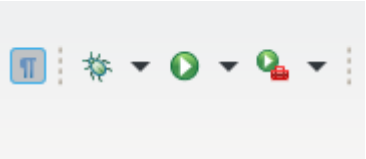
# Code generation from models (Model to Text)

1. Code generation from models (Model to Text)
2. Role of static profiles
3. Language xtend
4. Apply to simplified Python/Keras generator
5. **Test/debug an Eclipse plugin**
6. Revision / questions
7. Homework

# Debugging an Eclipse (workspace) plugin

- Eclipse only takes installed plugins into account
- But a ***new/2nd Eclipse*** instance will contain your workspace plugins
- Behaves like a normal Eclipse, runs in its own workspace

- Create a new Eclipse instance and run it
- Open the Debug Configurations dialog:
  - Menu **Run -> Debug configurations**
  - Via **Tool bar**



# Debug configuration dialog

**Create, manage, and run configurations**

Create a configuration to launch an Eclipse application in debug mode.

type filter text

- Acceleo Application
- AspectJ/Java Application
- AspectJ Load-Time Weaving Application
- C/C++ Application
- C/C++ Attach to Application
- C/C++ Container Launcher
- C/C++ Postmortem Debugger
- C/C++ Remote Application
- C/C++ Unit
- Eclipse Application
  - Launch Runtime Eclipse
  - New_configuration
  - test-environment
- GDB Hardware Debugging
- Java Applet
- Java Application
  - Main (1)
  - New_configuration (2)
- JUnit
- JUnit Plug-in Test

Filter matched 41 of 54 items

Name: New_configuration

Main | Arguments | Plug-ins | Configuration | Tracing | Environment | Common | Prototype

Launch with: all workspace and enabled target plug-ins

Default Start level: 4

Default Auto-Start: false

type filter text

| Plug-ins | Start Level | Auto-Start |
|---|---|---|

Select All

Deselect All

Add Working Set...

Add Required Plug-ins

Restore Defaults

Only show selected

0 out of 1534 selected

Include optional dependencies when computing required Plug-ins

Add new workspace Plug-ins to this launch configuration automatically

Validate Plug-ins automatically prior to launching

Validate Plug-ins

Show Command Line | Revert | Apply

Close | Debug

# Place a breakpoint, inspect variables

DesignerTraceMe | M2MTrafoUtil.ja

```
109     } else if (eObj instanceof Operation) {
110         return ((Operation) eObj).getClass_();
111     } else if (eObj instanceof Property) {
112         return ((Property) eObj).getClass_();
113     } else {
114         return null;
115     }
116 }
117
118 /**
119  * Obtain list of all available M2M transformations
120  * Restrict to those doing tracing. This is currently base
121  * library must be called "tracing" with a package M2M tra
122  */
123 public List<M2MTrafoChain> getM2MChain(EObject eObj) {
124     EList<M2MTrafoChain> list = new BasicEList<M2MTrafoCha
125     if (eObj != null) {
126         // load rules of registered Tracing model library
127         Element tracingPkg = ElementUtils.getQualifiedElem
128         if ((tracingPkg instanceof Package)) {
129             return M2MTrafoUtil.getTransformationChains((P
130         }
131     }
132     return list;
133 }
134
135 /**
136  * TODO: function does currently nothing
137  *
138  * @see org.eclipse.papyrus.moka.tracepoint.service.ITrace
```

Variables | Breakpoints | Modules

| Name | Value |
|---|---|
| no method return value | |
| this | DesignerTraceMechanism (id=922) |
| eObj | ClassImpl (id=924) |
| classifierBehavior | StateMachineImpl (id=1004) |
| collaborationUses | SubsetSupersetEObjectContainmentEList$Resolving<E> (i |
| eAnnotations | null |
| eContainer | PackageImpl (id=1022) |
| eFlags | -1769468 |
| eFlags | 256 |
| elementImports | EObjectContainmentWithInverseEList$Resolving<E> (id=1 |
| eStorage | Adapter[12] (id=1029) |
| generalizations | EObjectContainmentWithInverseEList$Resolving<E> (id=1 |
| interfaceRealizations | EObjectContainmentWithInverseEList$Resolving<E> (id=1 |
| name | "C1" (id=1032) |
| nameExpression | null |
| nestedClassifiers | EObjectContainmentEList$Resolving<E> (id=1036) |
| ownedAttributes | EObjectContainmentEList$Resolving<E> (id=1038) |

<Choose a previously entered expression>

org.eclipse.uml2.uml.internal.impl.ClassImpl@563f9766 (name: C1, visibility: <uns

# Stack trace

Debugger stopped on breakpoint, indicates line, shows code

Debug | Project Expl | Type Hierarchy

- Thread [main] (Suspended (breakpoint at line 125 in DesignerTraceMechanism)
  - DesignerTraceMechanism.getM2MTrafos(EObject) line: 125 (out of synch)
  - DesignerTraceMechanism.getTraceMechanismIDs(EObject) line: 51 (out of sy
  - TraceActionSelection$TraceMechanismsCP.getElements(Object) line: 181
  - CheckboxTableViewer(StructuredViewer).getRawChildren(Object) line: 949
  - CheckboxTableViewer(ColumnViewer).getRawChildren(Object) line: 704
  - CheckboxTableViewer(AbstractTableViewer).getRawChildren(Object) line: 10
  - CheckboxTableViewer(StructuredViewer).getFilteredChildren(Object) line: 85
  - CheckboxTableViewer(StructuredViewer).getSortedChildren(Object) line: 103
  - CheckboxTableViewer(AbstractTableViewer).internalRefreshAll(boolean) line:
  - CheckboxTableViewer(AbstractTableViewer).internalRefresh(Object, boolean

User can navigate to any element in the hierarchy

Some code changes (method signature) require a restart of the 2nd Eclipse instance

DesignerTraceMe | M2MTrafoUtil.ja

```
116 »   }¶
117 ¶
118 »   /**¶
119 »   ·* Obtain list of all available M2M transformations¶
120 »   ·* Restrict to those doing tracing. This is currently bas
121 »   ·* library must be called "tracing" with a package M2M tr
122 »   ·*/¶
123 »   public List<M2MTrafoChain> getM2MChain(EObject eObj) {¶
124 »   »   EList<M2MTrafoChain> list = new BasicEList<M2MTrafoCh
125 »   »   if (eObj != null) {¶
126 »   »   »   // load rules of registered Tracing model library
127 »   »   »   Element tracingPkg = ElementUtils.getQualifiedEle
128 »   »   »   if ((tracingPkg instanceof Package)) {¶
129 »   »   »   »   return M2MTrafoUtil.getTransformationChains(
130 »   »   »   }¶
131 »   »   }¶
132 »   »   return list;¶
133 »   }¶
134 ¶
135 »   /**¶
136 »   ·* TODO: function does currently nothing¶
137 »   ·*¶
138 »   ·* @see org.eclipse.papyrus.moka.tracepoint.service.ITrac
```

# Code generation from models (Model to Text)

1. **Code generation from models (Model to Text)**
2. **Role of static profiles**
3. **Language xtend**
4. **Apply to simplified Python/Keras generator**
5. **Test/debug an Eclipse plugin**
6. **Revision / questions**

# Revision – What we´ve learned …

- Motivation – why modeling?
- Domain specific vs. general purpose languages
- Textual vs. graphical modeling languages
- Meta-models
- Domain specific (modeling) languages – DSLs / DSMLs
  - Editor generation with xtext
- Domain specific modeling with UML
  - Extend the language – UML profiles
  - Restrict the language – profiles with OCL rules
  - Exploit the language for code generation – static profiles

# What is round-trip engineering?

**"The ability to automatically maintain the consistency of multiple, changing software artifacts, in software development environments/tools, is commonly referred to as round-trip engineering"**

- Related to traditional two software engineering disciplines:
  - Forward engineering: creating software from specifications
  - Reverse engineering: creating specifications from existing software
- Round-trip engineering adds synchronization of existing artifacts that evolved concurrently by incrementally updating each artifact to propagate changes made to the other artifact
- Round-trip generalizes both forward and reverse engineering

[Sendall04] S. Sendall and J. Küster. Taming model round-trip engineering. In *Proceedings of Workshop on Best Practices for Model-Driven Software Development*, Vancouver, Canada, 2004.

# Revision – We´ve learned …

- UML diagrams
  - Global functions – Use case diagram
  - Different modes/states – state machine diagram
  - Interactions modeling – sequence diagrams
  - Component-based modeling – composite structure diagram
- Practical use of UML in Eclipse/Papyrus
- Model transformations and code generation
  - OMG QVT and ATL
  - Code generation (running/debugging plugins)

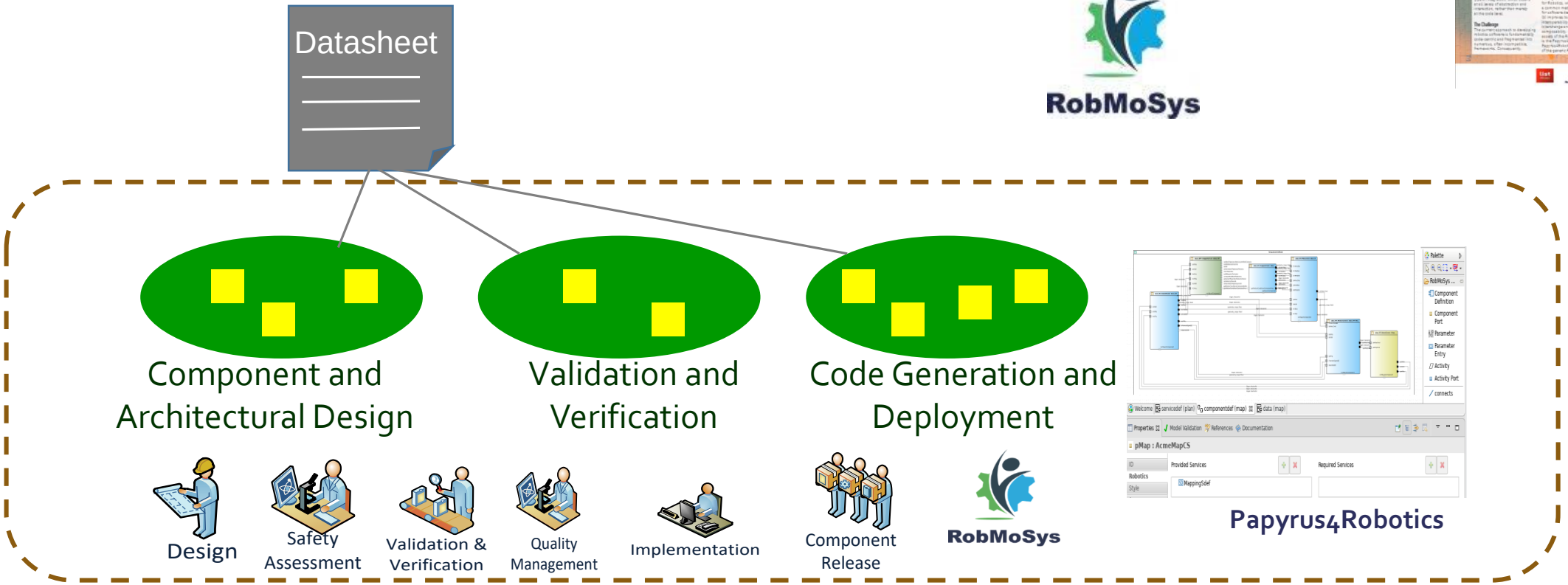
# Papyrus (for Robotics)

***Papyrus for Robotics*** – customization for the robotics domain

https://eclipse.org/papyrus/components/robotics

Youtube channel: https://www.youtube.com/c/PapyrusEclipseUML

RobMoSys aligned – European project https://robmosys.eu/



# The Robot Operating System (ROS)

- https://index.ros.org/doc/ros2/
- Set of software libraries and tools for building robot applications.
- Wide range (drivers, algorithms, visualization tools …), open source.
- ROS 1 was started in 2007
- ROS 2 – reduced footprint, based on DDS middleware, better real-time support
- microROS for resource-constrained systems

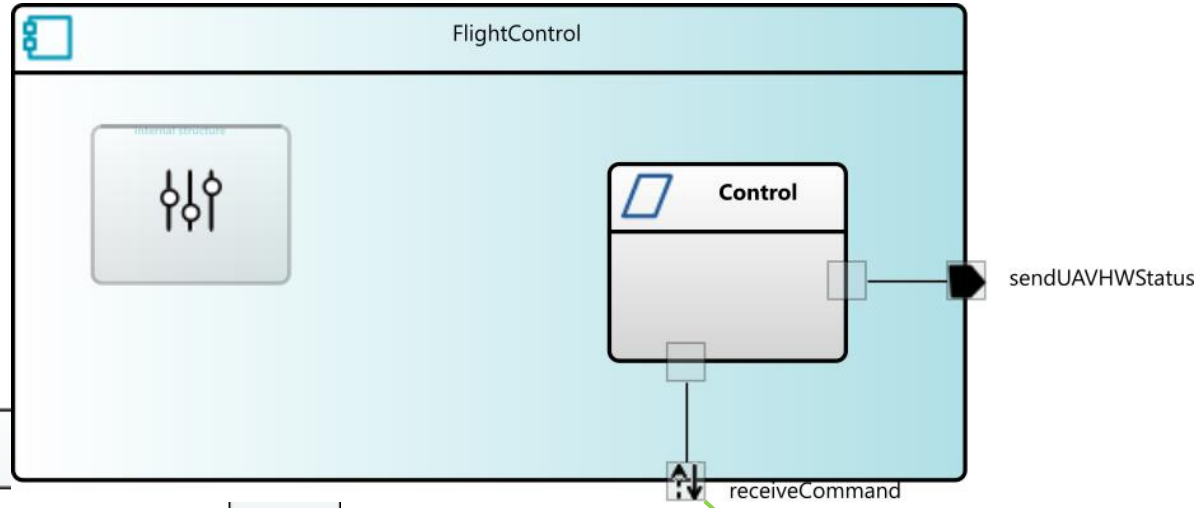
⇨ Growing number of companies migrating to ROS2
⇨ Papyrus for Robotics supports code generation for ROS2

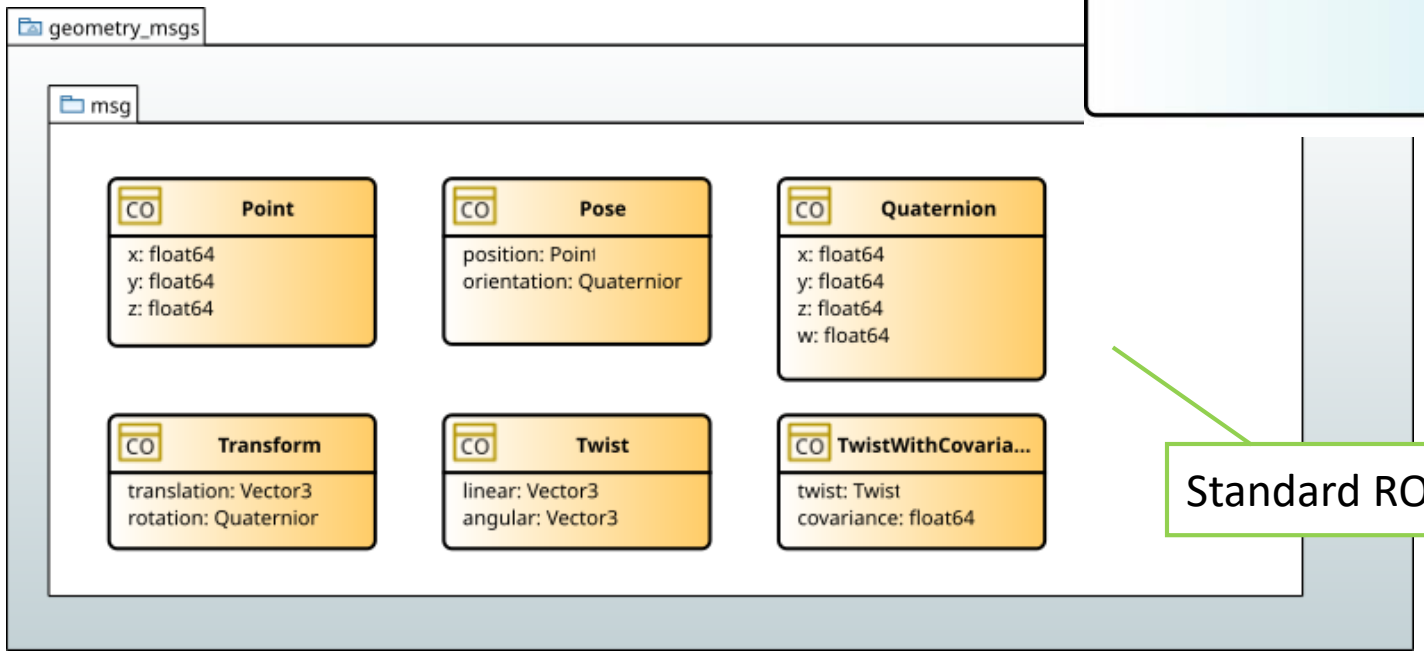
ROS-MDD

ROSin

# Examples in Papyrus for Robotics

- Flight control component
- Uses ROS service definitions

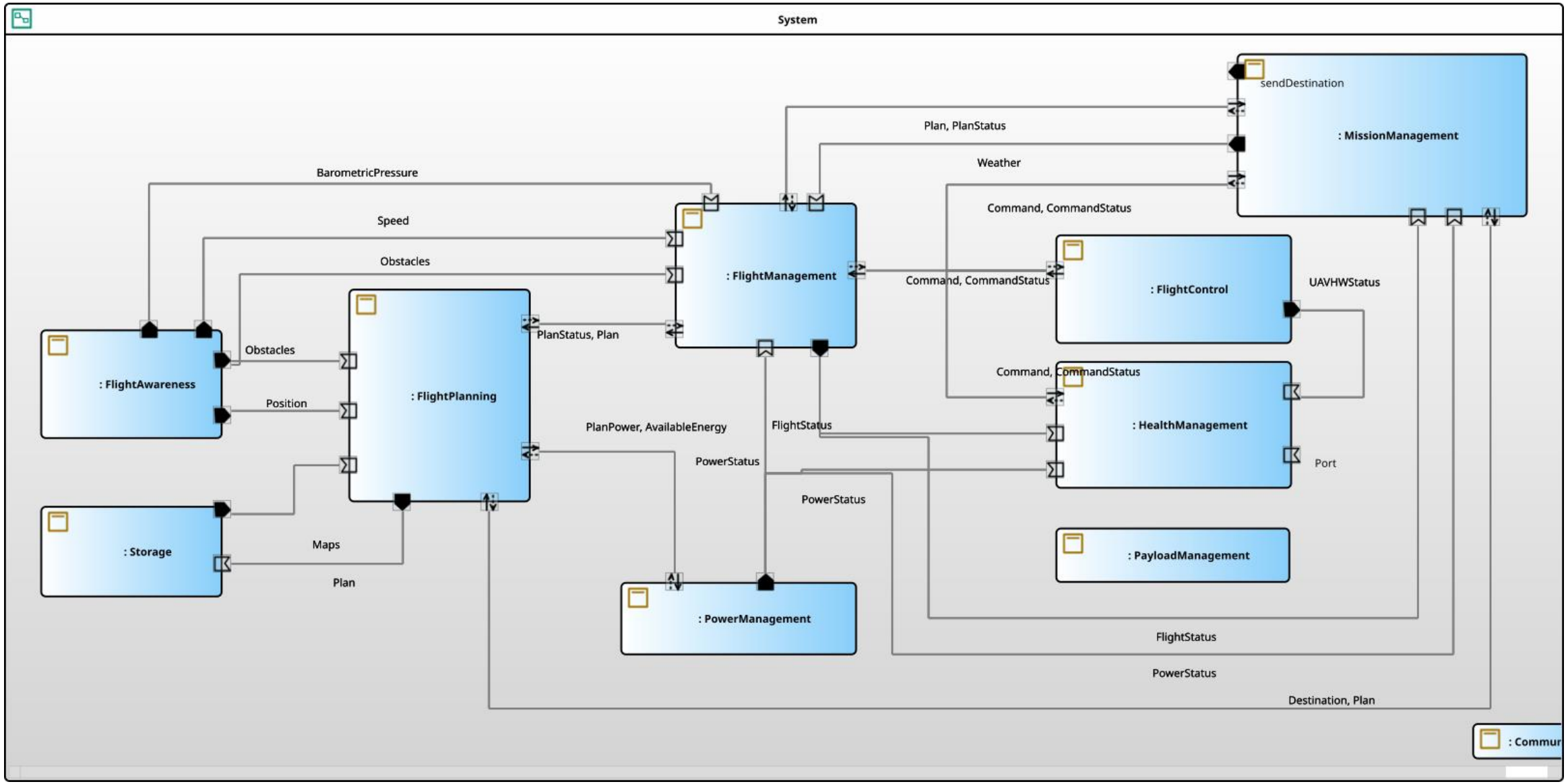



Port references ROS2 service

Standard ROS2 data types

# Example System model



# Feedback

- Which topic did you consider most important? (most interesting?)
- What did you understand / what was too difficult to understand?
- Too many topics, just right or something important was missing?
- Quality of course & material?
- Importance of modeling skills for getting hired?